

DSCI 523 Quiz 2

Tidy control flow

- `group_by(a, b)` marks groups; rows stay. `summarise()` → one row/group; grouped `mutate()` → rows retained. `n()` counts group. Ungroup before global operations.
- `mean()/max()/median()` propagate NA; use `na.rm = TRUE` inside function. `drop_na(x, y)` drops rows missing those columns; `drop_na()` checks all. Choose drop, replace, or preserve deliberately; detect missing with `is.na(x)`, never `x == NA`.
- `case_when(test ~ value, ..., TRUE ~ old)` applies rowwise; unmatched values otherwise become NA. Branch types must agree; convert factors with `as.character()` before text replacement.

```
df |> group_by(g) |> summarise(avg = mean(x, na.rm = TRUE), n = n())
df |> group_by(g) |> mutate(delta = x - first(x)) |> ungroup()
```

Output-shape quick rules

- `group_by(g)` |> `summarise(...)`: one row/group; grouped `mutate(...)`: original rows remain; `nest()`: one outer row/group + list-column data.
- `map_dbl(data, f)`: one scalar per item; `map(data, f)`: arbitrary objects/list-column. Only `unnest()` when the inner shapes are compatible; it may expand rows.

Mapping and nested data

- `map(.x, .f, ...)` returns a list; `map_dbl/int/lgl/chr()` requires one output of that type per item; `map_df()` combines rows as a tibble. ... supplies arguments to each call (`na.rm = TRUE`). Choose suffix from output shape; `map()` remains a list.
- `.f` may be a named function or anonymous function(`x`) ... / ~ ... (`x` current item, `.y` second input). Use `map*` when requested, not `across()`.
- `map2*` maps paired positions of two inputs; `pmap*` maps corresponding positions across inputs. Check lengths and output shape. **Optional**: multi-input mapping; prefer basic `map*` unless needed.
- A list-column holds one object per row. `nest()` keeps group keys and puts rows in data; `mutate(..., map(data, ...))` computes per nested object; `unnest(data)` expands rows—check shape.

```
nested <- df |> group_by(g) |> nest()
nested |> mutate(avg = map_dbl(data, ~ mean(.x$x, na.rm = TRUE)))
nested |> unnest(data)
map2_dbl(x, y, ~ .x + .y) # two inputs
pmap_dbl(list(x, y, z), function(a, b, c) a + b + c) # many
```

Functions, tests, errors

- `f <- function(x, n = 2) { ... }`; defaults apply only when omitted. Last expression is returned; `return(value)` exits early. Arguments are lazy (evaluated when used). Inner names mask outer names; prefer explicit inputs over mutable globals. Roxygen2 function contract: `@param`, `@return`, `@examples`; document interface, not each line.
- **Optional-Advanced**: manually handling `.... pkg::fun()` calls without attaching; `source()` loads a script.
- Validate at entry; `stop("useful message")` errors, `warning()` continues. `try()` permits continued execution after error; it does not validate input.
- TDD: specify normal, edge, and error behavior → implement → test again. Use `test_that()` with `expect_equal()` (numeric; tolerance only if allowed), `expect_identical()` (type/attributes), `expect_true/false()`, `expect_error()` / `expect_warning()` / `expect_output()`.
- Lab 3 sample variance (not `var()`): `sum((x - mean(x))^2) / (length(x) - 1)`; one numeric result; reject list/data-frame inputs.

```
mpg_to_kml <- function(mpg) {
  if (!is.numeric(mpg)) stop("mpg must be numeric")
  mpg * 0.425144
}
test_that("conversion and input contract", {
  expect_equal(mpg_to_kml(1), 0.425144)
  expect_error(mpg_to_kml("A"))
})
```

Tidy evaluation

- Data masking: verbs resolve bare names as columns. In reusable functions, `{ { col } }` captures/injects a bare column argument; equivalently capture with `enquo(col)` and inject `!!col`.
- String column name: `.data[[col]]` (validate one existing string name); outside variable: `.env$threshold`. These differ from bare masked columns.
- Computed output name: `:=` (e.g. `"{name}_mean" := ...`). ... passes arguments through only when contract is clear. **Optional**: elaborate .../tidy-eval interfaces.
- Validate inputs (data, column, type, parameter); fail clearly, then check output type, names, rows, and grouping.

```
mean_by <- function(data, group, value) {
  data |> group_by({{ group }}) |>
    summarise(mean_value = mean({{ value }}, na.rm = TRUE), .groups = "drop")
}
mean_col <- function(data, col) {
  stopifnot(is.character(col), length(col) == 1, !
    is.na(col), col %in% names(data))
  summarise(data, value = mean(.data[[col]], na.rm = TRUE))
}
```

```
# Add an argument to each mapped call: use ... or an anonymous function
map_df(data_entry, str_replace, pattern = "Cdn", replacement = "Canadian")
map_chr(month, ~ paste0(.x, ", 2020"))
```

```
# A nested tibble can hold a scalar and a more complex result per group
nested |> mutate(n = map_int(data, nrow),
  stats = map(data, ~ summarise(.x, avg = mean(value, na.rm = TRUE))))
```

- `map2(.x, .y, .f)` pairs corresponding positions; `.y` of length 1 can recycle. `pmap(list(...), .f)` maps matching positions across inputs; `pmap(df, .f)` is row-wise. These multi-input forms are **Optional** in Lecture 7.
- In the short formula function, `.` is the current item in the lecture examples; `.x/.y` are clearer for one/two inputs. For 3+ inputs use a named function(`a, b, c`) or `..1, ..2, ...` in formula form.
- `nest()` rows represent groups, not individual observations. `unnest()` can multiply rows; inspect `nrow()`, columns, and key duplication afterward.

Function and testing reminders

```
# Last expression returns; validate the contract at entry
f <- function(x, scale = 1) {
  if (!is.numeric(x)) stop("x must be numeric")
  x * scale
}

test_that("scales values and rejects invalid input", {
  expect_equal(f(c(2, 3), 2), c(4, 6))
  expect_error(f("2"))
})
```

- `expect_identical()` checks value/type/attributes; `expect_equal()` allows numeric tolerance; `expect_true/false()`, `expect_error()`, `expect_warning()`, `expect_output()` assert other contracts. Use tolerance only when rounding error is permitted.
- TDD: specify normal + boundary + invalid cases → implement → rerun all tests. Keep tests simple; tests can pass while missing cases.
- Lazy arguments are evaluated only when used; a function's local name masks an outer name, and unresolved names are looked up in enclosing environments. Pass changing dependencies explicitly rather than relying on globals.
- Roxygen contract: description + `@param` for each argument + `@return` + `@examples` (Worksheet 6 also shows `@export`). `pkg::fun()` avoids attaching a package; `source(path)` loads script definitions.

Tidy-eval interface chooser

```
# Bare column/expression from caller: embrace at the dplyr boundary
filter_rows <- function(data, condition) filter(data, {{ condition }})
filter_rows(penguins, body_mass_g > 3000)
```

```
# Dynamic result name: :=; string column: .data[[name]]
summary <- function(data, col, out) {
  stopifnot(is.character(col), length(col) == 1, col %in% names(data))
  summarise(data, "{out}" := mean(.data[[col]], na.rm = TRUE))
}
```

- Use `{{ arg }}` when a wrapper takes an unquoted column/expression. Equivalent explicit route: `q <- enquo(arg)` then use `!!q` inside the tidy verb.
- Use `.data[[name]]` for a string column name and `.env$limit` for an ordinary environment value; avoids a same-named column masking a local variable.
- Use `...` to forward multiple selectors/arguments unchanged to one verb (e.g. `select(data, ...)`). Put it last when callers should pass it positionally. Do not reuse the same dots as both a column expression and an unrelated selection; embrace separately named arguments instead.
- When the function must change a column's value type, inspect it with `pull({{ col }})/is.numeric()` and `stop()` before `mutate()`. For string-name APIs, validate one existing name and its type first. ""